

Exercise: Movie Watchlist Implementation

Difficulty: Beginner | Estimated Time: 40 minutes

Overview

In this exercise, you will implement a **Movie Watchlist** system using JavaScript. You will practice creating arrays of objects, managing data within arrays, searching, filtering, sorting, and displaying formatted results.

This exercise tests your understanding of:

1. Array manipulation (`push` , `splice`)
2. Searching arrays (`find` , `findIndex` , `filter`)
3. Sorting and slicing arrays
4. Functions and parameters
5. Basic loops and arithmetic

Question 1: Watchlist Setup

File: q1-watchlist-setup.js

Task 1.1: Create the Movies Array

Create an array called `movies` that contains **5 movie objects**. Each movie object must have the following properties:

- `id` — a unique number (1, 2, 3, 4, 5)
- `title` — a string (the movie's name)
- `genre` — a string (e.g. "Action", "Drama", "Sci-Fi", "Comedy", "Horror")
- `year` — a number (the release year)
- `rating` — a number out of 10 (e.g. 8.5)

You may choose any 5 movies you like.

Task 1.2: Create the Watchlist Array

Create an empty array called `watchlist` that will store movies added by the user.

Task 1.3: Create the addToWatchlist Function

1. Takes a `movieId` as a parameter.
2. Finds the movie from the `movies` array that matches the ID.
3. If the movie is already in the watchlist, do NOT add it again — return `false`.
4. Adds the movie object to the watchlist array.
5. Returns `true` if successful, or `false` if the movie was not found or was already added.

Task 1.4: Create the removeFromWatchlist Function

1. Takes a `movieId` as a parameter.
2. Finds the index of the movie in the watchlist array.
3. Removes it from the watchlist using the `.splice()` method.
4. Returns `true` if removed, or `false` if the movie was not in the watchlist.

Question 2: Search & Filter

File: q2-search-filter.js

Task 2.1: Create the `searchByTitle` Function

1. Takes a `keyword` as a parameter (a string).
2. Returns a new array of movies whose title contains the keyword. The search should be case-insensitive.
3. If no movies match, return an empty array.

Task 2.2: Create the `filterByGenre` Function

1. Takes a `genre` as a parameter (a string).
2. Returns a new array of all movies that match the given genre. The comparison should be case-insensitive.
3. If no movies match, return an empty array.

Task 2.3: Create the `getTopRated` Function

1. Takes a `count` parameter (a number).
2. Returns a new array of the top `count` movies sorted by rating (highest first).
3. If `count` is greater than the total number of movies, return all movies sorted by rating.

Question 3: Display Functions

File: q3-display.js

Task 3.1: Create the `getWatchlistCount` Function

Returns the total number of movies currently in the watchlist.

Task 3.2: Create the `getAverageRating` Function

1. Takes an array of movie objects as a parameter.
2. If the array is empty, return `0`.
3. Loops through the array and adds up all the `rating` values.
4. Divides the total by the number of movies.
5. Returns the average rounded to 1 decimal place.

Task 3.3: Create the `displayWatchlist` Function

1. If the watchlist is empty, prints `"Your watchlist is empty."` using `console.log()`.
2. Otherwise, prints `"My Watchlist:"` using `console.log()`.
3. Loops through the watchlist and prints each movie in this format:
`- Title (Year) - Rating/10`
4. At the end, prints the average rating by calling your `getAverageRating` function:
`Average Rating: X.X/10`

Example Output:

```
My Watchlist:
- Inception (2010) - 8.8/10
- The Dark Knight (2008) - 9.0/10
Average Rating: 8.9/10
```

Scoring

The assignment is graded out of **10 marks**. Each successful test case is worth **1 mark**.

#	Topic	Tests
Q1	Watchlist Setup	4
Q2	Search & Filter	3
Q3	Display Functions	3
Total		10